

Java Class

A class is a blueprint that defines data and behavior for objects. It groups related fields and methods in a single unit. Memory for instance members is allocated when an object is created, while static members are allocated when the class is loaded into memory.

- Acts as a template to create objects with shared structure.
 - Does not occupy memory for fields until instantiation.
 - Can contain fields, methods, constructors, nested classes and interfaces.
-

Existing Classes

Definition

Using an existing class means creating objects of a predefined class and using its methods and properties instead of writing new code. It promotes **code reusability**.

Example

```
class Student
{
    void display()
    {
        System.out.println("Hello");
    }
}

public class Main
{
    public static void main(String args[])
    {
        Student s = new Student();
        s.display();
    }
}
```

```
}  
}
```

Advantages

- Code Reusability
 - Easy Maintenance
 - Saves Development Time
-

Class Inheritance

Definition

Inheritance is the process by which one class acquires the properties and methods of another class.

The existing class is called the **Base Class (Parent/Super Class)**.

The new class is called the **Derived Class (Child/Sub Class)**.

Syntax

```
class Parent  
{  
}  
  
class Child extends Parent  
{  
}
```

Types of Inheritance

1. Single Inheritance

A
|
B

```
class Animal
{
    void eat()
    {
        System.out.println("Eating");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Barking");
    }
}
```

2. Multilevel Inheritance

A
|
B
|
C

Example

```
class A
{
    void display()
    {
        System.out.println("Class A");
    }
}
```

```
}  
  
class B extends A  
{  
}  
  
class C extends B  
{  
}
```

3. Hierarchical Inheritance



Example

```
class Animal  
{  
}  
  
class Dog extends Animal  
{  
}  
  
class Cat extends Animal  
{  
}
```

Choosing Base Class

Definition

A **Base Class** should contain only common properties and methods that can be shared by all derived classes.

Example



Instead of writing `eat()` separately in every class, place it in the **Animal** class.

Guidelines

- Keep only common members.
- Avoid unnecessary variables.
- Base class should be generic.
- Child class should contain specialized features.

Multiple Levels of Inheritance

Definition

When a derived class becomes the base class for another class, it is called **Multilevel Inheritance**.

Diagram

A

↓

B

↓

C

Example

```
class A
{
    void show()
    {
        System.out.println("A");
    }
}

class B extends A
{
}

class C extends B
{
}
```

Abstraction through Abstract Classes

Definition

An **Abstract Class** is a class declared using the **abstract** keyword.

- Cannot be instantiated.
- May contain abstract and normal methods.
- Used to achieve abstraction.

Syntax

```
abstract class Animal
{
```

```
    abstract void sound();

    void sleep()
    {
        System.out.println("Sleeping");
    }
}
```

Example

```
abstract class Animal
{
    abstract void sound();
}

class Dog extends Animal
{
    void sound()
    {
        System.out.println("Bark");
    }
}
```

Advantages

- Provides abstraction
- Code Reusability
- Better Design

Abstract Class vs Interface

Abstract Class	Interface
Uses <code>abstract</code> keyword	Uses <code>interface</code> keyword
Can have constructors	No constructors
Can have normal methods	Traditionally only abstract methods (modern Java also allows default/static methods)

Abstract Class	Interface
Supports partial abstraction	Primarily supports abstraction
A class can extend only one abstract class	A class can implement multiple interfaces

Packages

A **Package** is a collection of related classes and interfaces. It helps organize programs and avoids naming conflicts.

Advantages

- Organizes code
- Prevents name conflicts
- Access Protection
- Easy Maintenance

Creating Package

```
package mypack;
```

Import Package

```
import java.util.Scanner;
```

User-defined Package

```
package student;  
  
public class Test
```

```
{  
}
```

Using Package

```
import student.Test;
```

Standard Packages

Package	Purpose
java.lang	Core classes
java.util	Collections, Scanner
java.io	File Handling
java.net	Networking
java.awt	GUI

CLASSPATH

Definition

CLASSPATH is an environment variable that tells the JVM where to find user-defined classes and packages.

Example

```
CLASSPATH=C:\Java\Programs
```

Advantages

- Locates classes

- Locates packages
 - Helps JVM execute programs
-

Concept of Interface

Definition

An **Interface** is a blueprint of a class that contains method declarations. It is used to achieve abstraction and support multiple inheritance.

Syntax

```
interface Animal
{
    void sound();
}
```

Example

```
interface Animal
{
    void sound();
}

class Dog implements Animal
{
    public void sound()
    {
        System.out.println("Bark");
    }
}
```

Advantages

- Multiple Inheritance
- Loose Coupling
- Abstraction
- Better Program Design

Interface vs Abstract Class

Interface	Abstract Class
Uses <code>implements</code>	Uses <code>extends</code>
Supports multiple inheritance	Single inheritance only
No constructors	Constructors allowed
Primarily abstraction	Partial abstraction

Questions

1. Explain Object-Oriented Programming concepts (Encapsulation, Inheritance, Polymorphism, Abstraction) with respect to Java, giving one code example for each. (7-10 Marks)

Object-Oriented Programming (OOP) is a programming paradigm that organizes programs using **objects and classes**. Java is a fully object-oriented language (except for primitive data types) and is based on four main principles:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

1. Encapsulation

Encapsulation is the process of wrapping data (variables) and methods (functions) into a single unit called a **class**. Data is protected using **private** access

modifiers and accessed through **getter and setter methods**.

```
class Student
{
    private int age;

    public void setAge(int age)
    {
        this.age = age;
    }

    public int getAge()
    {
        return age;
    }
}

public class Main
{
    public static void main(String args[])
    {
        Student s = new Student();
        s.setAge(20);

        System.out.println(s.getAge());
    }
}
```

Output

20

2. Inheritance

Inheritance is the process by which one class acquires the properties and methods of another class using the **extends** keyword. It promotes code reusability.

```
class Animal
{
    void eat()
    {
        System.out.println("Eating");
    }
}

class Dog extends Animal
{
    void bark()
    {
        System.out.println("Barking");
    }
}

public class Main
{
    public static void main(String args[])
    {
        Dog d = new Dog();

        d.eat();
        d.bark();
    }
}
```

Output

```
Eating
Barking
```

3. Polymorphism

Polymorphism means "one interface, many forms." It allows the same method name to perform different tasks.

Java supports:

- Method Overloading (Compile-time Polymorphism)
- Method Overriding (Run-time Polymorphism)

Compile-Time Polymorphism

Compile-time polymorphism is the type of polymorphism in which the method to be executed is determined by the compiler during compilation. It is achieved through **method overloading**.

Run-Time Polymorphism

Run-time polymorphism is the type of polymorphism in which the method to be executed is determined at runtime based on the object. It is achieved through **method overriding**.

Example (Method Overloading)

```
class Add
{
    int sum(int a, int b)
    {
        return a + b;
    }

    int sum(int a, int b, int c)
    {
        return a + b + c;
    }
}

public class Main
{
    public static void main(String args[])
    {
        Add obj = new Add();

        System.out.println(obj.sum(5,6));

        System.out.println(obj.sum(2,3,4));
    }
}
```

```
}  
}
```

Output

```
11  
9
```

4. Abstraction

Abstraction is the process of hiding implementation details and showing only the essential features of an object. It is achieved using **abstract classes** and **interfaces**.

```
abstract class Animal  
{  
    abstract void sound();  
}  
  
class Dog extends Animal  
{  
    void sound()  
    {  
        System.out.println("Bark");  
    }  
}  
  
public class Main  
{  
    public static void main(String args[])  
    {  
        Dog d = new Dog();  
  
        d.sound();  
    }  
}
```

Output

Bark

2. Write a Java program to demonstrate access modifiers (`private` , `protected` , `public` , `default`) across two classes in the same and different packages. (10 Marks)

Access modifiers control the visibility and accessibility of classes, methods, and variables in Java.

There are four access modifiers:

- `private`
- `default`
- `protected`
- `public`

Access Modifier Table

Modifier	Same Class	Same Package	Subclass (Different Package)	Different Package
private	✓	X	X	X
default	✓	✓	X	X
protected	✓	✓	✓	X
public	✓	✓	✓	✓